

LABORATORIO di Analisi Numerica

Aritmetica di macchina e instabilità degli algoritmi

Ángeles Martínez Calomardo

Dipartimento di Matematica, Informatica e Geoscienze

Laurea Triennale in Intelligenza Artificiale e Data Analytics

A.A. 2026–2027

Aritmetica di macchina e propagazione degli errori

- ▶ Abbiamo visto che il numero x non si rappresenta esattamente nel calcolatore ma si immagazzina come $\text{fl}(x)$ con un errore relativo massimo pari a $u = \frac{1}{2}B^{1-t}$, con $t = 53$ nello standard IEEE-754r.
- ▶ Si può dimostrare che

$$\epsilon_{x,y}^{\oplus} \leq \left| \frac{x}{x+y} \right| \epsilon_x + \left| \frac{y}{x+y} \right| \epsilon_y$$

$$\epsilon_{x,y}^{\otimes} \approx \epsilon_x + \epsilon_y$$

$$\epsilon_{x,y}^{\ominus} \leq |\epsilon_x - \epsilon_y|$$

dove ϵ_x e ϵ_y sono tali che $\epsilon_x, \epsilon_y \leq u$

- ▶ Le operazioni macchina prodotto e divisione introducono un errore dell'ordine della precisione di macchina.
- ▶ Con la somma (sottrazione) non si può garantire che il risultato dell'operazione sia affetto da un errore relativo piccolo. In particolare l'errore per la somma è grande quando $x \approx -y$ (**Cancellazione numerica**).

Cancellazione numerica e stabilità di un algoritmo

Definizione

*Un metodo numerico (formula, algoritmo) si dice **stabile** se non propaga gli errori (inevitabili) dovuti alla rappresentazione dei numeri nel calcolatore. Altrimenti si dice **instabile**.*

- ▶ La cancellazione numerica genera delle formule instabili.
- ▶ Per evitare i problemi legati alla cancellazione numerica occorre trasformare le formule in altre numericamente più stabili.
- ▶ La stabilità è un concetto legato all'algoritmo usato per risolvere un determinato problema.

Esempio di algoritmo instabile

Dato $x^2 + 2px - q$, con $p^2 + q \geq 0$ eseguiamo un primo algoritmo Matlab che valuta la radice via:

$$y = -p + \sqrt{p^2 + q}. \quad (1)$$

- ▶ $p^2 + q \geq 0$ implica radici reali.
- ▶ Potenzialmente instabile per $p \gg q > 0$ a causa della sottrazione tra p e $\sqrt{p^2 + q}$ (cancellazione).

Valutiamo la radice con un secondo algoritmo stabile via razionalizzazione di (1):

$$\begin{aligned} y &= -p + \sqrt{p^2 + q} = \frac{(-p + \sqrt{p^2 + q})(p + \sqrt{p^2 + q})}{(p + \sqrt{p^2 + q})} \\ &= \frac{q}{(p + \sqrt{p^2 + q})} \end{aligned} \quad (2)$$

Codice stabilità : algoritmo 1

Salviamo il seguente codice in `radicesecgrado.m`.

```
p=1000; q=0.0180000000081; sol=0.9*10^(-5);  
  
% ALGORITMO 1  
s=p ^ 2;  
t=s+q;  
if t >=0  
    u=sqrt(t);  
else  
    fprintf( '\n \t [RADICI COMPLESSE] ');  
end  
s1=-p+u;
```

Codice stabilità : algoritmo 2

```
% ALGORITMO 2
s=p ^ 2;
t=s+q;
if t >=0
    u=sqrt(t);
else
    fprintf( '\n \t [RADICI COMPLESSE] ');
end
v=p+u;
t1=q/v;
```

Codice stabilità : stampa risultati

```
fprintf( '\n \t [ALG.1]: %10.16f', s1 );  
fprintf( '\n \t [ALG.2]: %10.16f', t1 );  
if (sol ~= 0)  
    rerr1 =abs(s1-sol)/abs(sol);  
    rerr2=abs(t1-sol)/abs(sol);  
    fprintf( '\n \t [REL.ERR.ALG.1]: %2.2e', rerr1 );  
    fprintf( '\n \t [REL.ERR.ALG.2]: %2.2e', rerr2 );  
end
```

Come previsto, il secondo algoritmo si comporta notevolmente meglio del primo, che compie un errore relativo dell'ordine di circa 10^{-9} . Infatti:

```
>> radicesecgrado  
  
[ALG.1]: 0.0000089999999773  
[ALG.2]: 0.0000090000000000  
[REL.ERR.ALG.1]: 2.52e-09  
[REL.ERR.ALG.2]: 0.00e+00  
>>
```

Eseguiamo un codice Matlab che valuti le successioni $\{u_n\}$, $\{z_n\}$, definite rispettivamente come

$$\begin{cases} s_1 = 1, s_2 = 1 + \frac{1}{4} \\ u_1 = 1, u_2 = 1 + \frac{1}{4} \\ s_{n+1} = s_n + \frac{1}{(n+1)^2} \\ u_{n+1} = \sqrt{6s_{n+1}} \end{cases}$$

e

$$\begin{cases} z_1 = 1, z_2 = 2 \\ z_{n+1} = 2^{n-\frac{1}{2}} \sqrt{1 - \sqrt{1 - 4^{1-n} \cdot z_n^2}} \end{cases} \quad (3)$$

che *teoricamente* convergono a π .

Implementiamo poi la successione, diciamo $\{y_n\}$, che si ottiene *razionalizzando* (3), cioè moltiplicando numeratore e denominatore di

$$z_{n+1} = 2^{n-\frac{1}{2}} \sqrt{1 - \sqrt{1 - 4^{1-n} \cdot z_n^2}}$$

per

$$\sqrt{1 + \sqrt{1 - 4^{1-n} \cdot z_n^2}}$$

e calcoliamo u_m , z_m e y_m per $m = 2, 3, \dots, 40$ (che teoricamente dovrebbero approssimare π).

Infine disegniamo in un unico grafico l'andamento dell'errore relativo di u_n , z_n e y_n rispetto a π aiutandoci con l'help di Matlab relativo al comando `semilogy`.

Calcolo π : metodo 1

In seguito scriviamo un'implementazione di quanto richiesto commentando i risultati. Si salvi in un file `pigreco.m` il codice

```
% SEQUENZE CONVERGENTI "PI GRECO".
```

```
% METODO 1.
```

```
s(1)=1; u(1)=1;
```

```
s(2)=1.25; u(2)=s(2);
```

```
for n=2:40
```

```
    s(n+1)=s(n)+(n+1)^(-2);
```

```
    u(n+1)=sqrt(6*s(n+1));
```

```
end
```

```
rel_err_u=abs(u-pi)/pi;
```

```
fprintf('\n');
```

Calcolo π : metodo 2



```
% METODO 2.  
format long  
z(1)=1;  
z(2)=2;  
for n=2:40  
    c=(4^(1-n)) * (z(n))^2; inner_sqrt=sqrt(1-c);  
    z(n+1)=(2^(n-0.5))*sqrt( 1-inner_sqrt );  
end  
rel_err_z=abs(z-pi)/pi;  
  
fprintf( '\n' );
```

Calcolo π : metodo 3



```
% METODO 3.  
y(1)=1;  
y(2)=2;  
for n=2:40  
    num=(2^(1/2)) * abs(y(n));  
    c=(4^(1-n)) * (y(n))^2;  
    inner_sqrt=sqrt(1-c);  
    den=sqrt(1+inner_sqrt);  
    y(n+1)=num/den;  
end  
rel_err_y=abs(y-pi)/pi;
```

Calcolo π : plots

```
% SEMILOGY PLOT.  
semilogy(1:length(u),rel_err_u,'k. ');  
hold on;  
semilogy(1:length(z),rel_err_z,'m+ ');  
semilogy(1:length(y),rel_err_y,'ro ');  
hold off;
```

Di seguito digitiamo sulla shell di Matlab/Octave

```
>> pigreco
```

Plot risultati

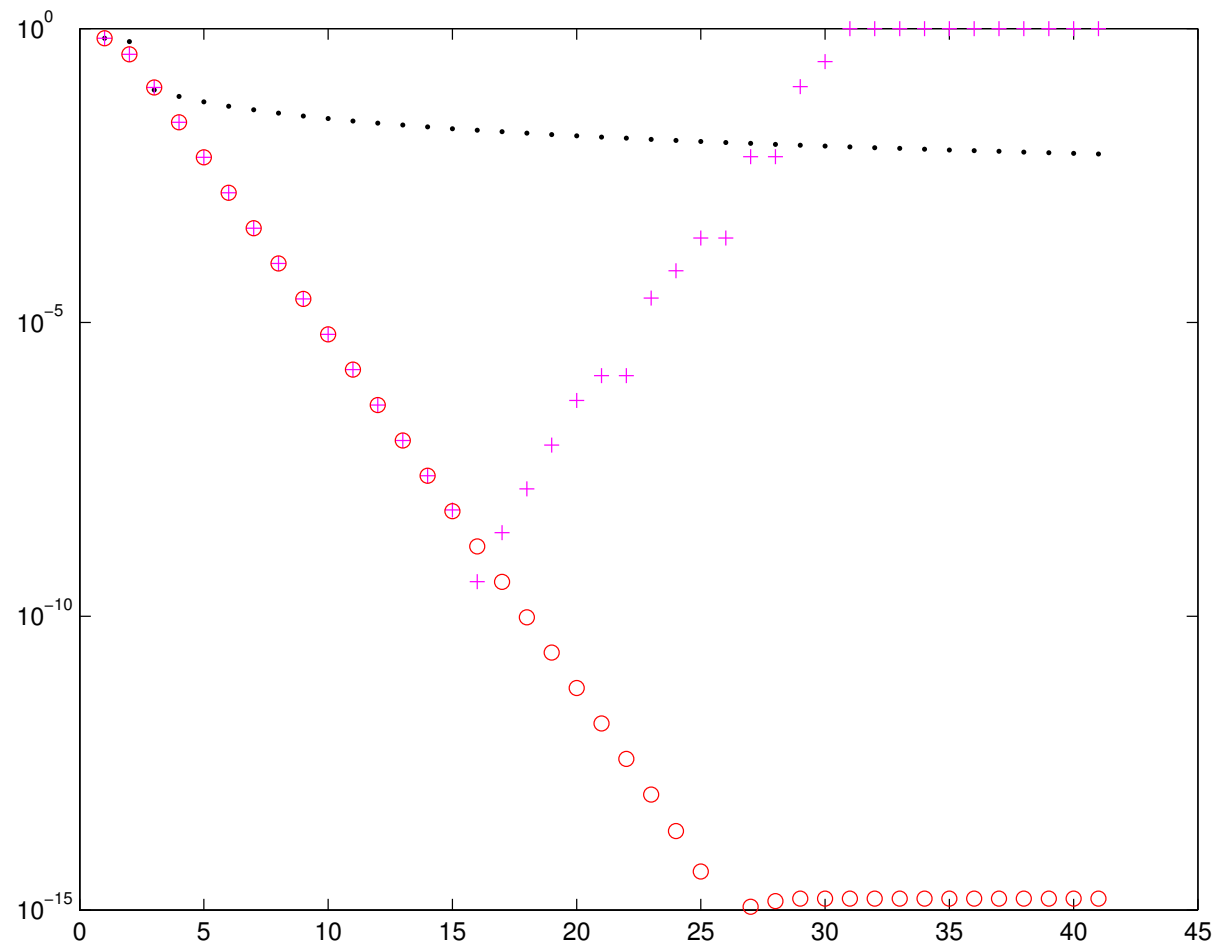


Figure 1: Errore relativo commesso con le 3 successioni, rappresentate rispettivamente da ., + e o.

- ▶ La prima successione converge molto lentamente a π , la seconda diverge mentre la terza converge velocemente a π .
- ▶ Per alcuni valori $\{z_n\}$ e $\{y_n\}$ coincidono per alcune iterazioni per poi rispettivamente divergere e convergere a π . Tutto ciò è naturale poichè le due sequenze sono analiticamente (ma non numericamente) equivalenti.
- ▶ Dal grafico dell'errore relativo, la terza successione, dopo aver raggiunto errori relativi prossimi alla precisione di macchina, si assesta ad un errore relativo di circa 10^{-15} .

L'algoritmo 2 in dettaglio

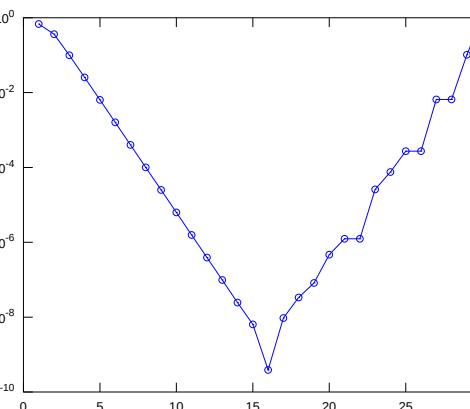
Nell'approssimare il valore di π con la formula ricorsiva

$$z_2 = 2$$

$$z_{n+1} = 2^{n-0.5} \sqrt{1 - \sqrt{1 - 4^{1-n} z_n^2}}, \quad n = 2, 3, \dots,$$

si ottiene la seguente successione di valori (dove si è posto $c = 4^{1-n} z_n^2$).

$n+1$	c	$1 - \sqrt{1-c}$	z_{n+1}	$\frac{ z_{n+1} - \pi }{\pi}$
...	...	...	...	...
10	1.505e-04	7.529e-05	3.14157294036	6.27e-06
11	3.764e-05	1.882e-05	3.14158772527	1.57e-06
12	9.412e-06	4.706e-06	3.14159142150	3.92e-07
13	2.353e-06	1.176e-06	3.14159234561	9.80e-08
14	5.882e-07	2.941e-07	3.14159257654	2.45e-08
15	1.470e-07	7.353e-08	3.14159263346	6.41e-09
16	3.676e-08	1.838e-08	3.14159265480	3.88e-10
17	9.191e-09	4.595e-09	3.14159264532	2.63e-09
18	2.297e-09	1.148e-09	3.14159260737	1.47e-08
19	5.744e-10	2.872e-10	3.14159291093	8.19e-08
...	...	...	...	...
28	2.220e-15	1.110e-15	3.16227766016	6.58e-03
29	5.551e-16	3.330e-16	3.46410161513	1.03e-01
30	1.665e-16	1.110e-16	4.00000000000	2.73e-01
31	5.551e-17	0.000e+00	0.00000000000	1.00e+00
32	0.000e+00	0.000e+00	0.00000000000	1.00e+00



Una successione ricorrente.

Consideriamo la successione $\{I_n\}$ definita da

$$I_n = e^{-1} \int_0^1 x^n e^x dx \quad (4)$$

► $n = 0$: $I_0 = e^{-1} \int_0^1 e^x dx = e^{-1}(e^1 - 1).$

► integrando per parti

$$\begin{aligned} I_{n+1} &= e^{-1} \left(x^{n+1} e^x \Big|_0^1 - (n+1) \int_0^1 x^n e^x dx \right) \\ &= 1 - (n+1) I_n. \end{aligned}$$

► $I_n > 0$, decrescente e si prova che $I_n \rightarrow 0$ come $1/n$.

Esercizio.



Calcolare I_n per $n = 1, \dots, 100$:

- mediante la successione *in avanti*

$$\begin{cases} I_0 = e^{-1}(e^1 - 1) \\ I_{n+1} = 1 - (n+1)I_n. \end{cases} \quad (5)$$

- mediante la successione *all'indietro*

$$\begin{cases} I_{200} = 0 \\ I_{n-1} = (1 - I_n)/n. \end{cases}$$

Si noti che se $I_{n+1} = 1 - (n+1)I_n$ allora $I_n = (1 - I_{n+1})/(n+1)$ e quindi $I_{n-1} = (1 - I_n)/n$.

Successione ricorrente IMPLEMENTAZIONE in Matlab



- ▶ Scrivere il codice in un file `succricorrente.m`.
- ▶ Salvare i valori calcolati in due vettori chiamati `s` (successione in avanti) e `t` (successione all'indietro).
- ▶ Per la successione in avanti partire da I_1 anziché da I_0 .
- ▶ Per la successione all'indietro partire da $I_{200} = 0$.
- ▶ Creare un grafico semilogaritmico con i valori $I_1, I_2, \dots, I_{100}$ calcolati dai due algoritmi che, ricordiamo, sono matematicamente equivalenti ma non numericamente equivalenti.

Successione ricorrente in Matlab

Scriviamo il codice in un file succricorrente.m.

```
% SUCCESSIONE RICORRENTE.  
clear all;  
% SUCCESSIONE "s_n".  
s(1)=exp(-1);  
for n=1:99  
    s(n+1)=1-(n+1)*s(n);  
end  
% SUCCESSIONE "t_n".  
M=200;  
t=zeros(M,1); % INIZIALIZZAZIONE "t".  
for n=M:-1:2  
    j=n-1;  
    t(j)=(1-t(n))/n;  
end
```

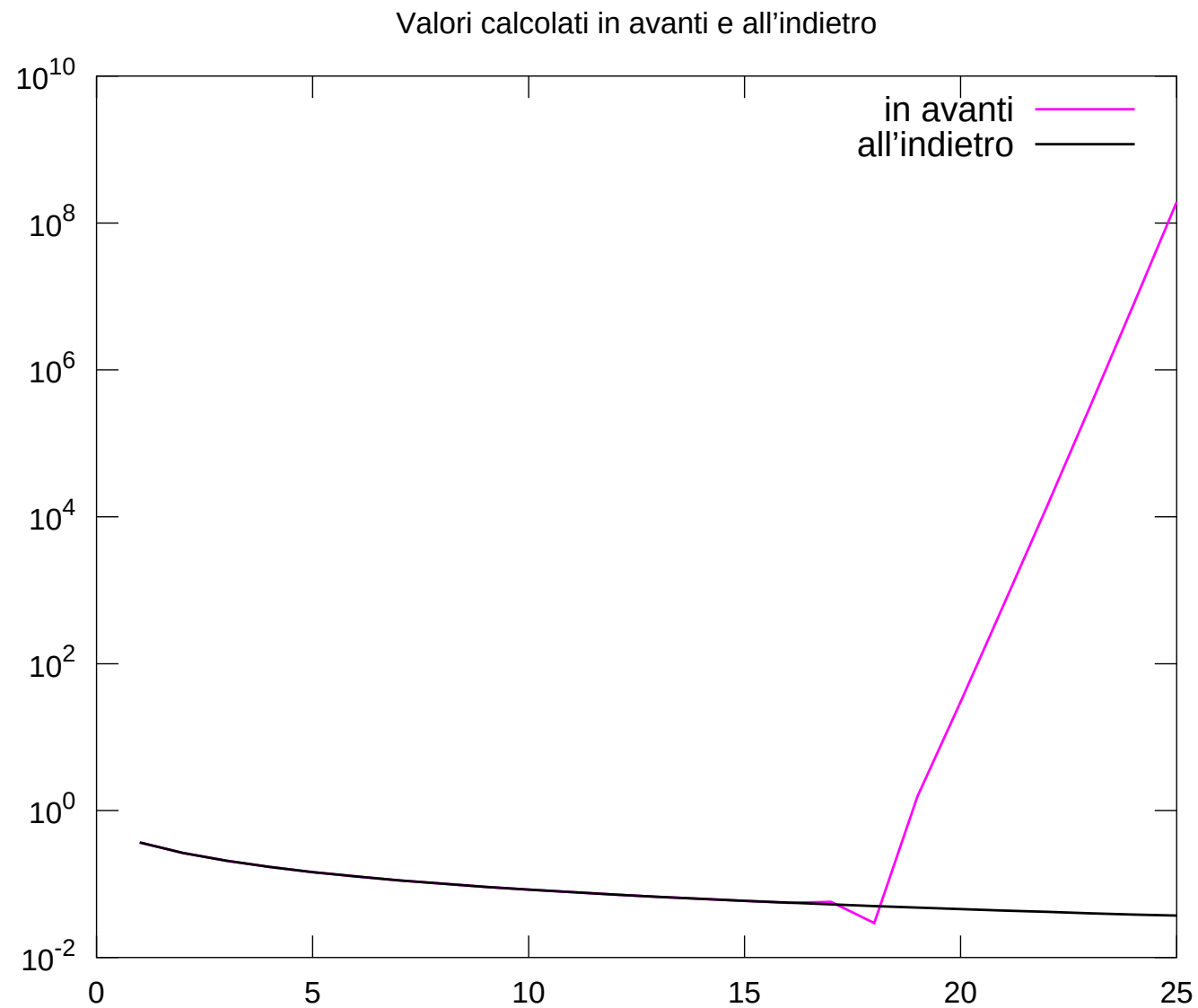
Successione ricorrente in Matlab

```
%% PLOT SEMI-LOGARITMICO.  
clf;  
semilogy(1:length(s),abs(s),'k-'); hold on;  
semilogy(1:length(s),abs(t(1:length(s))),'m-');  
hold off;
```

Di seguito digitiamo sulla shell di Matlab/Octave

```
>> succricorrente
```

Plot risultati



Instabilità della formula ricorsiva

La formula $I_n = 1 - n I_{n-1}$ è instabile, quindi amplifica l'errore ad ogni passo.

Infatti, nel calcolatore

$$(I_n + \varepsilon_n) = 1 - n(I_{n-1} + \varepsilon_{n-1}).$$

Sottraendo dalla precedente equazione la relazione $I_n = 1 - nI_{n-1}$ si può quantificare l'errore:

$$\varepsilon_n = -n \varepsilon_{n-1}, \quad \text{e per induzione} \quad |\varepsilon_n| = n! |\varepsilon_0|$$

Il fattore $n!$ amplifica l'errore di rappresentazione iniziale (su I_0), ε_0 .

► **Esempio.** Nel calcolo di I_{20} l'errore è $\varepsilon_{20} = 20! \varepsilon_0 \approx 2.7 \cdot 10^2$.

Formula alternativa stabile

Per l'errore al passo $n - 1$ si trova

$$\varepsilon_{n-1} = \frac{-1}{n} \varepsilon_n.$$

Partendo da m

$$|\varepsilon_{m-1}| = \frac{|\varepsilon_m|}{m}, \quad |\varepsilon_{m-2}| = \frac{|\varepsilon_m|}{m(m-1)}, \quad \dots, \quad |\varepsilon_{m-k}| = \frac{|\varepsilon_m|}{m(m-1) \cdots (m-k+1)}.$$

- ▶ La produttoria al denominatore abbatte rapidamente l'errore iniziale!
- ▶ Per esempio, per calcolare I_{25} partendo da $I_{40} = 0.5$, l'errore iniziale $|\varepsilon_{40}| < 0.5$ verrebbe abbattuto di un fattore

$$40 \cdot 39 \cdots 27 \cdot 26 = 5.2602 \cdot 10^{22}$$

.